



towboot

a Multiboot-compatible bootloader for
EFI-based x86 systems

Niklas Sombert

2025-07-10

1 Introduction

Goals

Motivation

2 Background

Technologies used

Alternatives

3 Implementation and Usage

Boot process

Configuration

Memory Management

towbootctl

Workarounds

Changes made to other software

4 Testing

Components

Bootloader

5 Conclusion

Testing

Summary

Further Improvements

Code

- 1 Introduction
 - Goals
 - Motivation

Goals

- UEFI application
- Multiboot-compatible bootloader

Motivation

- firmware of current devices follows the UEFI standard
- Multiboot as the interface between kernel and bootloader
- hhuOS
 - Multiboot-compatible
 - licensing reasons
 - whole operating system

② Background

Technologies used

UEFI

Rust

Multiboot

x86

ACPI

SMBIOS

ELF

TOML

Alternatives

- Unified Extensible Firmware Interface¹
- Tianocore²
- Open Virtual Machine Firmware³

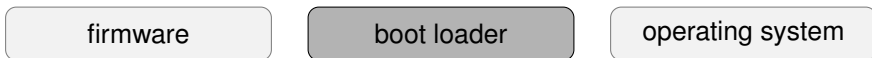


Figure: the boot process (approximately)

¹ *Unified Extensible Firmware Interface (UEFI) Specification, Version 2.8 Errata B, May 2020.* UEFI Forum, 2020. URL: <https://uefi.org/sites/default/files/resources/UEFI%20Spec%202.8B%20May%202020.pdf> (visited on 10/13/2020).

² *tianocore/edk2.* URL: <https://github.com/tianocore/edk2> (visited on 01/01/2021).

³ *OVMF.* tianocore/tianocore.github.io. URL: <https://github.com/tianocore/tianocore.github.io/wiki/OVMF> (visited on 01/01/2021).

(without a Compatibility Support Module)

- relocatable Portable Executables
- loaded by the firmware (or another application)
- same CPU mode as the firmware
- `BootXXXX` variables
 - EFI System Partition
- removable media
 - `\EFI\BOOT\BOOTarch.EFI`

- `efi_main` function, parameters:
 - System Table
 - Boot and Runtime Services
- opaque handles
- protocols with UUIDs
 - may have a revision
 - may be installed by third-party software


```
1  #![no_main]
2  #![no_std]
3
4  use uefi::{prelude::*, println};
5
6  #[entry]
7  fn efi_main(
8      _handle: Handle, _system_table: SystemTable<Boot>,
9  ) -> Status {
10     uefi::helpers::init().unwrap();
11     println!("Hello, World!");
12
13     Status::SUCCESS
14 }
```

- Loaded Image Protocol
 - parent application, device, path, command line, size, memory address
- Simple File System Protocol / File Protocol
 - at least FAT12/16/32
- Simple Text Input Protocol / Simple Text Output Protocol
- Graphics Output Protocol⁴
 - framebuffer, usable after boot
- many more
 - Universal Graphics Adapter Protocol, block access, mouse, ...

⁴Replacing VGA, GOP implementation for UEFI. UEFI Summer Plugfest. 2011. URL: https://uefi.org/sites/default/files/resources/UPFS11_P4_UEFI_GOP_AMD.pdf (visited on 10/25/2020).

```
1  // get information about the way we were loaded
2  let loaded_image = system_table.boot_services()
3      .open_protocol_exclusive::<LoadedImage>(handle)
4      .expect("Failed to open loaded image protocol");
5
6  // get the load options
7  let load_options: CString16 = loaded_image.load_options_as_cstr16()
8      .unwrap().into();
```

Technologies used: Rust⁵

- system programming language
- low-level code, high-level abstractions
- type- and memory-safety (excluding `unsafe`)
- cargo

⁵ *Rust Programming Language*. URL: <https://www.rust-lang.org/> (visited on 01/01/2021).

- `targets i686-unknown-uefi` and `x86_64-unknown-uefi` (and `aarch64-unknown-uefi`)⁶
 - Tier 2 (built in the CI)
 - `std` works partially
- `uefi` crate⁷

⁶*The rustc book*. Chap. Platform Support. URL:

<https://doc.rust-lang.org/nightly/rustc/platform-support.html> (visited on 06/23/2023).

⁷Gabriel Majeri. *uefi*. crates.io. URL: <https://crates.io/crates/uefi> (visited on 12/04/2020).

- standard for kernels and bootloaders
- from the GNU GRUB⁸ project
 - GNU kernel: gnumach⁹ (for GNU HURD¹⁰)
- two major versions

⁸GNU GRUB. GNU. URL: <https://www.gnu.org/software/grub/> (visited on 10/13/2020).

⁹gnumach. GNU. URL: <https://www.gnu.org/software/hurd/microkernel/mach/gnumach.html> (visited on 12/05/2020).

¹⁰GNU Hurd. GNU. URL: <https://www.gnu.org/software/hurd/> (visited on 01/01/2021).

¹¹Bryan Ford and Erich Stefan Boleyn. *Multiboot Specification version 0.6.96*. Free Software Foundation, Inc. URL: <https://www.gnu.org/software/grub/manual/multiboot/multiboot.html> (visited on 10/13/2020).

¹²Bryan Ford and Erich Stefan Boleyn. *Multiboot2 Specification version 2.0*. GNU. Free Software Foundation, Inc. URL: <https://www.gnu.org/software/grub/manual/multiboot2/multiboot.html> (visited on 10/13/2020).

- load kernel and modules (ELF or flat binary)
 - ELF or flat binary
 - header: static information
 - module alignment
 - require passing memory or video information
 - preferred video mode
 - binary loading and entry
 - Multiboot 2: also support for UEFI entry and Boot Services
 - additional flags

- pass information about the system
 - struct, EAX and EBX registers
 - memory (upper, lower, map)
 - kernel command line
 - modules (and command line)
 - symbol
 - bootloader name
 - framebuffer
 - legacy stuff (boot device, drives, config table, APM, VBE)
 - on Multiboot 2: SMBIOS, ACPI, UEFI, network
 - machine state
 - 32-bit Protected Mode
 - no interrupts
 - no paging
 - no PAE (in practice)
 - Multiboot 2: also native UEFI mode

- multiboot crate¹³, PR¹⁴
- multiboot2 crate¹⁵, PR¹⁶

¹³Gerd Zellweger. *multiboot*. crates.io. URL: <https://crates.io/crates/multiboot> (visited on 01/04/2021).

¹⁴Niklas Sombert. *Add some functionality for using this in bootloaders (Pull Request 8)*. gz/rust-multiboot. URL: <https://github.com/gz/rust-multiboot/pull/8> (visited on 01/04/2021).

¹⁵Philipp Oppermann. *multiboot2*. crates.io. URL: <https://crates.io/crates/multiboot2> (visited on 06/23/2023).

¹⁶Niklas Sombert. *Add a builder to multiboot2 (Pull Request 133)*. rust-osdev/multiboot2. URL: <https://github.com/rust-osdev/multiboot2/pull/133> (visited on 06/23/2023).

- i686 (32-bit) and x86_64 (64-bit)
- multiple modes¹⁷
 - Long Mode
 - 64-Bit Mode
 - Compatibility Mode (32-Bit)
 - Legacy Mode
 - Protected Mode (32-Bit or 16-Bit)
 - Virtual-8086 Mode (16-Bit)
 - Real Mode (16-Bit)
- Alternatives:
 - MIPS (Multiboot 2)
 - Itanium, ARM, RISC-V (UEFI)

¹⁷ AMD64 Architecture Programmers Manual. 2020. Chap. 1.2 Modes of Operation. URL: <https://www.amd.com/system/files/TechDocs/40332.pdf> (visited on 01/04/2021).

- Multiboot machine state requires many x86-specifics
 - switch from native mode to 32-bit Protected Mode
 - may have to switch through Compatibility Mode
- memory
 - Real Mode (8086): 16-bit pointers, 24-bit address bus, 1MB, memory hole, lower 640KB
 - 286: 24-bit address bus: 1MB to approx 10MB
 - i686: 4GB virtually, physically (more with PAE)
 - x86_64: physically 256TB memory
 - *segmentation*

- in Real Mode, segment registers are offsets

```
1     mov [ax], 0xab ; writes to *(ds * 16 + ax)
2     mov es:[ax], 0xab ; similar, but with es
3     jmp 0xabc ; actually jumps to cs * 16 + 0xabc ("near-jump")
4     ; also FS, GS and SS, for push and pop
```

- they're settable, usually

```
1     mov ds, 0x12
2     jmp 0xde:0xabc ; cs can only be set with a "far-jump"
3     ; or a "far-call" or "far-ret"
```

- in Protected Mode, segment registers are indices into the Global Descriptor Table
- for a very simple OS with just one application it might look like this:

index	content
0	NULL
1	system code
2	system data
3	application code
4	application data

- information contained in these Segment Descriptors:
 - base, limit, access, flags
 - not layed out sequentially!
- `x86 crate`¹⁸ has a builder

¹⁸Gerd Zellweger. *x86*. *crates.io*. URL: <https://crates.io/crates/x86> (visited on 06/30/2023).

```
1  let code_segment_builder = SegmentDescriptorBuilder::code_descriptor(  
2      0, u32::MAX, CodeSegmentType::ExecuteRead,  
3  );  
4  let code_segment: Descriptor = code_segment_builder  
5      .present().limit_granularity_4kb().db().finish();  
6  let data_segment_builder = SegmentDescriptorBuilder::data_descriptor(  
7      0, u32::MAX, DataSegmentType::ReadWrite,  
8  );  
9  let data_segment: Descriptor = data_segment_builder  
10     .present().limit_granularity_4kb().db().finish();  
11  let gdt_array = [Descriptor::NULL, code_segment, data_segment];  
12  let gdt = DescriptorTablePointer::new_from_slice(&gdt_array);
```

```
1  unsafe { x86::irq::disable();  
2      x86::dtables::lgdt(&gdt);  
3      x86::dtables::lidt(&DescriptorTablePointer::default()); // invalid  
4      asm!("push 0x08", // code segment  
5          "lea rbx, [2f]",  
6          "push rbx",  
7          "retfq", // overwrite CS  
8          "2:", // compatibility mode, yay.  
9          ".code32",  
10         "mov eax, 0x10", // data segment  
11         "mov ds, eax",  
12         "mov es, eax",  
13         "mov fs, eax",  
14         "mov gs, eax",  
15         "mov ss, eax",
```

Technologies used: x86: code (3/3)

```
1      "mov ecx, cr0",
2      "and ecx, ~(1<<31)", // disable paging
3      "or ecx, 1", // enable protected mode
4      "mov cr0, ecx",
5
6      "mov ecx, cr4",
7      "and ecx, ~(1<<5)", // disable PAE
8      "mov cr4, ecx",
9
10     "mov ecx, 0xC0000080",
11     "rdmsr",
12     "and eax, ~(1<<8)", // disable long mode
13     "wrmsr",
14 );
15 }
```


- Advanced Configuration and Power Interface
- replacement for the Advanced Power Management
- currently developed by the UEFI Forum
- firmware provides pointer to the RSDP via the system table
 - contains a checksum, version info and a pointer to the Root System Descriptor Table
 - the bootloader passes it to the kernel as part of the Multiboot info struct
 - on BIOS systems, the OS would need to scan the memory for it
- `acpi crate`¹⁹

¹⁹Isaac Woods. *acpi*. crates.io. URL: <https://crates.io/crates/acpi> (visited on 07/04/2025).

- System Management BIOS
- part of Intel's (now obsolete) Wired for Management Baseline Specification²⁰
- allows the OS to discover hardware
- firmware provides pointer to the Entry Point via the system table
 - contains version info and a pointer to the structure table
 - the bootloader passes it to the kernel as part of the Multiboot info struct
 - on BIOS systems, the OS would need to scan the memory for it
- `dmidecode` crate²¹

²⁰ *Intel Wired for Management*. Intel Corporation. URL: <https://web.archive.org/web/20050828044750/http://www.intel.com/technology/computing/wfm.htm> (visited on 06/11/2024).

²¹ Jonathan Creekmore. *dmidecode*. crates.io. URL: <https://crates.io/crates/dmidecode> (visited on 07/04/2025).

- Executable and Linkable Format²²
- support needed to be Multiboot-compliant
- header
- Program Header
- Section Header
- `goblin` crate²³

²² [Wikipedia contributors](https://en.wikipedia.org/w/index.php?title=Executable_and_Linkable_Format&oldid=992446183). *Executable and Linkable Format* — *Wikipedia, The Free Encyclopedia*. 2020. URL: https://en.wikipedia.org/w/index.php?title=Executable_and_Linkable_Format&oldid=992446183 (visited on 01/04/2021).

²³ [m4b et al. *goblin*. crates.io](https://crates.io/crates/goblin). URL: <https://crates.io/crates/goblin> (visited on 01/04/2021).

Technologies used: TOML

- Tom's Obvious Minimal Language²⁴
- format for configuration files
- simple, yet type-safe and well-defined
- `toml` crate²⁵

²⁴Tom Preston-Werner. *TOML: Tom's Obvious Minimal Language*. URL: <https://toml.io/> (visited on 10/14/2020).

²⁵Eric Huss and Alex Crichton. *toml*. [crates.io](https://crates.io/crates/toml). URL: <https://crates.io/crates/toml> (visited on 01/04/2021).

- GNU GRUB²⁶
 - widely used bootloader
 - supports various platforms and operating systems
 - menu, can edit entries interactively
 - complex scripting language
 - de-facto reference implementation of Multiboot
- Syslinux²⁷
 - very lightweight bootloader
 - supports multiple platforms and some operating systems
 - current release 2014
- Limine²⁸
 - supports various platforms and operating systems
 - also has an own boot protocol

²⁶ GNU GRUB.

²⁷ The Syslinux Project. URL: <https://www.syslinux.org/> (visited on 01/02/2021).

²⁸ Limine. URL: <https://limine-bootloader.org/> (visited on 06/08/2023).

3 Implementation and Usage

- Boot process

- Configuration

 - NVRAM

 - Command line parameters

 - Configuration file

 - quirks

 - Paths

- Memory Management

 - Stack

 - Heap

 - Whole pages

- towbootctl

- Workarounds

- Changes made to other software

 - multiboot crates

 - uefi-rs

 - acpi

 - hhuOS

 - miniarg

 - multiboot12

- ① initialize the bootloader
- ② determine what to boot
 - ① parse the command line parameters
 - ② parse the configuration file
 - ③ display a menu
- ③ boot the operating system
 - ① load the kernel
 - ② load the modules
 - ③ configure the graphics output
 - ④ prepare the information to pass to the kernel
 - ⑤ deinitialize most parts of the bootloader
 - ⑥ jump to the kernel's entry point

- binary encoding: CBOR²⁹ or JSON³⁰ (for instance)
- EFI variable
- requires tooling
- bound to a system
- firmware might “forget” configuration

²⁹Carsten Bormann. *Concise Binary Object Representation*. URL: <https://cbor.io> (visited on 10/14/2020).

³⁰Douglas Crockford. *Introducing JSON*. URL: <https://www.json.org/> (visited on 10/14/2020).


```
FS0:\> towboot.efi -help
```

Usage:

-config Load the specified configuration file instead of the default one.
-kernel Don't load a configuration file, instead boot the specified kernel.
-logLevel Set the log level. (This only applies if '-kernel' is specified.)
-module Load a module with the given args. Can be specified multiple times.
-quirk Enable a specific quirk. (Only applies when loading a kernel.)
-help Displays all available options and how to use them.
-version Displays the version of towboot

- `towboot.efi -kernel "kernel.elf quiet" -module "module1.bin -verbose" -module "module2.bin -quiet"`
- `towboot.efi -config config.toml`
- file names or arguments may contain spaces or quotes
- `BootXXXX` variables
- shell or from another bootloader
- boot manager in the firmware usually not very user-friendly
- firmware might “forget” configuration

```
1 default = "entry1" 12
2 timeout = 10 13
3 # log_level defaults to 'info' 14
4 15
5 [entries] 16
6     [entries.entry1] 17
7         name = "Operating System" 18
8         image = "kernel.elf" 19
9         argv = "quiet" 20
10        quirks = ["KeepResolution"] 21
11
```

```
[[entries.entry1.modules]]
    image = "module1.bin"
    argv = "--verbose"
```

```
[[entries.entry1.modules]]
    image = "module2.bin"
    argv = "--quiet"
```

- easy to read and modify by hand
- most bootloaders do this
- `towboot.toml`, placed in top-level directory of the ESP

- specifics of how the kernel is loaded can be configured at runtime
- either in the `quirk` key of a kernel entry or via the `-quirk` command line option
- available quirks:
 - `DontExitBootServices`: do not exit Boot Services
 - `ForceElf`: always treat the kernel as an ELF file
 - `ForceOverwrite`: ignore the memory map when loading the kernel
 - `KeepResolution`: ignore the kernel's preferred resolution
 - `ModulesBelow200Mb`: keep allocations for modules below 200 MB

- paths are
 - absolute if they start with a volume identifier (`fs? :`)
 - relative to the volume to boot itself is on if they start with a backslash
 - relative to the configuration file
- paths relative to the UEFI shell's current working directory are not supported, yet
- paths for kernel and modules given on the commandline can't contain spaces
 - use a configuration file for this

- local variables
- works always
- size needs to be known at compile time
- used for most runtime data, for example `MultibootInfo`
- tracked by `rustc` at compile time

- Rust's `alloc::alloc::alloc` is bound to UEFI's `allocate_pool`
- tracked both by `rustc` at compile time and by the firmware at runtime
- used for everything with a dynamic size and no further requirements

Memory Management: Whole pages

- can be allocated to specific address (used for the kernel)
- custom `mem::Allocation wrapper (impl Drop)`
- tracked by the firmware at runtime
- also used for modules

- a library that allows
 - reading and writing the configuration file and command line options
 - most of this is actually in another crate called `towboot_config`
 - building images containing towboot, a configuration file, kernels and their modules
 - booting those images using QEMU or Bochs
 - downloading OVMF for this
- this library is used by the build tooling (`xtask`) and the integration tests
- a command line utility that exposes these functions and bundles the towboot binaries

- stack probes³¹ broken with Rust 1.49 (fixed with 1.51, at least)
- initially no support for floating point operations³²³³

³¹*probestack.rs*. [rust-lang/compiler-builtins](https://github.com/rust-lang/compiler-builtins/blob/63ccaf11f08fb5d0b39cc33884c5a1a63f547ace/src/probestack.rs). URL: <https://github.com/rust-lang/compiler-builtins/blob/63ccaf11f08fb5d0b39cc33884c5a1a63f547ace/src/probestack.rs> (visited on 01/04/2021).

³²*request: add fmodf for x86_64-unknown-uefi (Issue 434)*. [rust-lang/compiler-builtins](https://github.com/rust-lang/compiler-builtins/issues/434). URL: <https://github.com/rust-lang/compiler-builtins/issues/434> (visited on 06/07/2024).

³³*request: add fmod and fmodf for i686-unknown-uefi (Issue 490)*. [rust-lang/compiler-builtins](https://github.com/rust-lang/compiler-builtins/issues/490). URL: <https://github.com/rust-lang/compiler-builtins/issues/490> (visited on 06/07/2024).

- `multiboot crate`³⁴ was designed for use in kernels
- added support for parsing the header, for setting values in `MultibootInfo` and tests³⁵
- most difficult part: integration with Rust's and UEFI's memory management
- the same goes for `multiboot2`³⁶³⁷

³⁴Zellweger, *multiboot*.

³⁵Sombert, *Add some functionality for using this in bootloaders (Pull Request 8)*.

³⁶Oppermann, *multiboot2*.

³⁷Sombert, *Add a builder to multiboot2 (Pull Request 133)*.

- some improvements to the build process
 - `build-std-features`³⁸
 - `compiler-builtins`³⁹

³⁸Niklas Sombert. *Use build-std-features instead of rlibc (Pull Request 298)*. [rust-osdev/uefi-rs](https://github.com/rust-osdev/uefi-rs). URL: <https://github.com/rust-osdev/uefi-rs/pull/298> (visited on 06/07/2024).

³⁹Niklas Sombert. *Don't explicitly require compiler-builtins(-mem) (Pull Request 534)*. [rust-osdev/uefi-rs](https://github.com/rust-osdev/uefi-rs). URL: <https://github.com/rust-osdev/uefi-rs/pull/534> (visited on 06/07/2024).

Changes made to other software: acpi

- make more fields readable⁴⁰

⁴⁰Niklas Sombert. *rsdp: Make whole RSDP readable (Pull Request 179)*. rust-osdev/acpi. URL: <https://github.com/rust-osdev/acpi/pull/179> (visited on 06/07/2024).

Changes made to other software: hhuOS

- kernel assumed the Multiboot structs to be before the kernel in memory
- PR⁴¹ to copy them to BSS very early
- kernel also looks for the free memory after data passed by the bootloader

⁴¹[Niklas Sombert](#). *Copy the Multiboot info struct recursively to BSS before enabling paging (Pull Request 19)*. hhuOS/hhuOS. URL: <https://github.com/hhuOS/hhuOS/pull/19> (visited on 01/09/2021).

Changes made to other software: miniarg

- parses strings such as `program -foo "bar baz"`
- `no_std-compatible`
- released to crates.io⁴²
- not specific to UEFI or Multiboot

⁴²Niklas Sombert. *miniarg*. crates.io. URL: <https://crates.io/crates/miniarg> (visited on 12/05/2020).

Changes made to other software: multiboot12

- abstraction over `multiboot` and `multiboot2` crates
- currently just laying around on GitHub⁴³

⁴³Niklas Sombert. *multiboot12*. URL: <https://github.com/Ytvwld/multiboot12> (visited on 06/30/2023).

- 4 Testing
 - Components
 - Bootloader

Components

- unit tests for `multiboot`, `multiboot2` and `miniarg`
- not (yet?) for `multiboot12`

Bootloader

- automated and manual end-to-end tests
- Multiboot specs⁴⁴⁴⁵ contain example kernels
- had to be modified to print to serial
- booting a whole operating system
- virtual machine

⁴⁴Ford and Boleyn, *Multiboot Specification version 0.6.96*.

⁴⁵Ford and Boleyn, *Multiboot2 Specification version 2.0*.

5 Conclusion

Testing

Summary

Further Improvements

Code

- successfully tested with the test kernels, hhuOS⁴⁶, GNU HURD⁴⁷, Lemon OS⁴⁸, HelenOS⁴⁹ and NetBSD⁵⁰
- tests with FlingOS⁵¹, OpenIndiana⁵², Fiwix⁵³ and 9front⁵⁴ failed

⁴⁶ *hhuOS*. URL: <https://hhuos.github.io/> (visited on 01/01/2021).

⁴⁷ *GNU Hurd*.

⁴⁸ *Lemon OS*. URL: <https://lemonos.org/> (visited on 06/30/2023).

⁴⁹ *HelenOS*. URL: <http://www.helenos.org/> (visited on 06/30/2023).

⁵⁰ *NetBSD*. URL: <https://netbsd.org/> (visited on 06/30/2023).

⁵¹ *Edward Nutting*. *FlingOS*. URL: <http://www.flingos.co.uk/> (visited on 01/01/2021).

⁵² *OpenIndiana*. URL: <https://www.openindiana.org/> (visited on 06/30/2023).

⁵³ *Fiwix*. URL: <https://www.fiwix.org/> (visited on 07/07/2025).

⁵⁴ *9front*. URL: <https://9front.org/> (visited on 07/07/2025).

Conclusion: Summary

- UEFI's abstractions make writing a bootloader very straight-forward
- `uefi-rs` maps them to Rust
- Rust support is good and is getting better
- Multiboot requires x86-specifics

Conclusion: Further Improvements

- Secure Boot⁵⁵
 - enabled by default on most modern systems
 - changes to kernels and the kernel file format needed to be secure
- 64-bit, UEFI-unaware kernels
- other CPU architectures?
- load improvements
- file paths

⁵⁵Richard Wilkins and Brian Richardson. *UEFI Secure Boot In Modern Computer Security Solutions*. 2013. URL: https://uefi.org/sites/default/files/resources/UEFI_Secure_Boot_in_Modern_Computer_Security_Solutions_2019.pdf (visited on 10/25/2020).

hhuOS/towboot

a bootloader for Multiboot kernels on UEFI systems
written in Rust



3

Contributors



2

Issues



20

Stars



2

Forks



<https://github.com/hhuOS/towboot>

⑥ Appendix: cat

Goal

Setup

Run

- ① takes a file name as parameter
- ② opens the file
- ③ reads the file
- ④ prints its contents


```
1  # install Rust
2  curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
3  # install the target
4  rustup target add x86_64-unknown-uefi
5  # install uefi-run
6  cargo install uefi-run
```

Appendix: Setup: project

```
1 $ cargo new --bin cat
2 Creating binary (application) `cat` package
3 note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/
4 $ cd cat
5 $ cargo add uefi@0.31 --features alloc,global_allocator,logger,panic_handler
6 Updating crates.io index
7   Adding uefi v0.31 to dependencies
8     Features as of v0.31.0:
9       + alloc
10      + global_allocator
11      + log-debugcon
12      + logger
13      + panic_handler
14      - qemu
15      - unstable
```

Appendix: launching the application

```
1 $ cargo build --target x86_64-unknown-uefi
2 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
3 $ uefi-run target/x86_64-unknown-uefi/debug/cat.efi
4 UEFI Interactive Shell v2.2
5 EDK II
6 UEFI v2.70 (EDK II, 0x00010000)
7 Mapping table
8     FS0: Alias(s):F0a:;BLK0:
9         PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
10     BLK1: Alias(s):
11         PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
12 Press ESC in 1 seconds to skip startup.nsh or any other key to continue
13 Starting UEFI application...
14 Hello, World!
15 FS0:\>
```